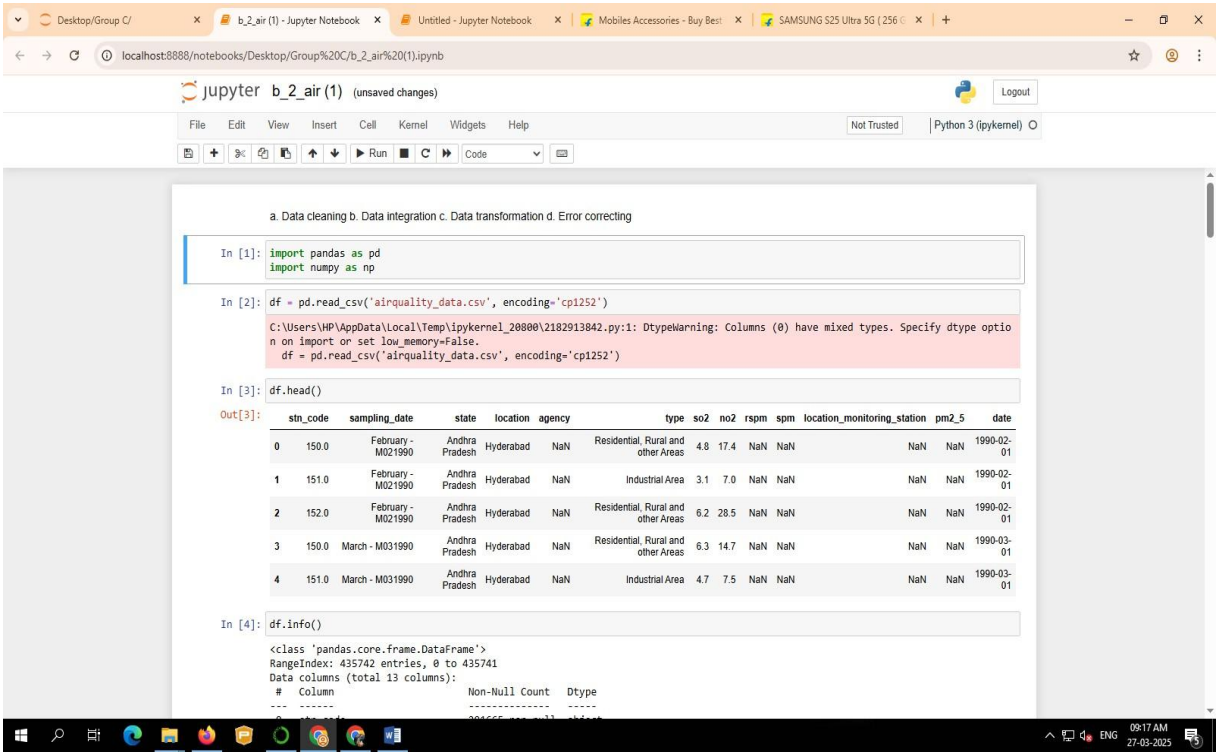
GROUP B) 2)



a. Data cleaning b. Data integration c. Data transformation d. Error correcting

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
df = pd.read_csv('airquality_data.csv', encoding='cp1252')
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_20800\2182913842.py:1: DtypeWarning: Columns (8) have mixed types. Specify dtype optio
n on import or set low_memory=False.
  df = pd.read_csv('airquality_data.csv', encoding='cp1252')
```

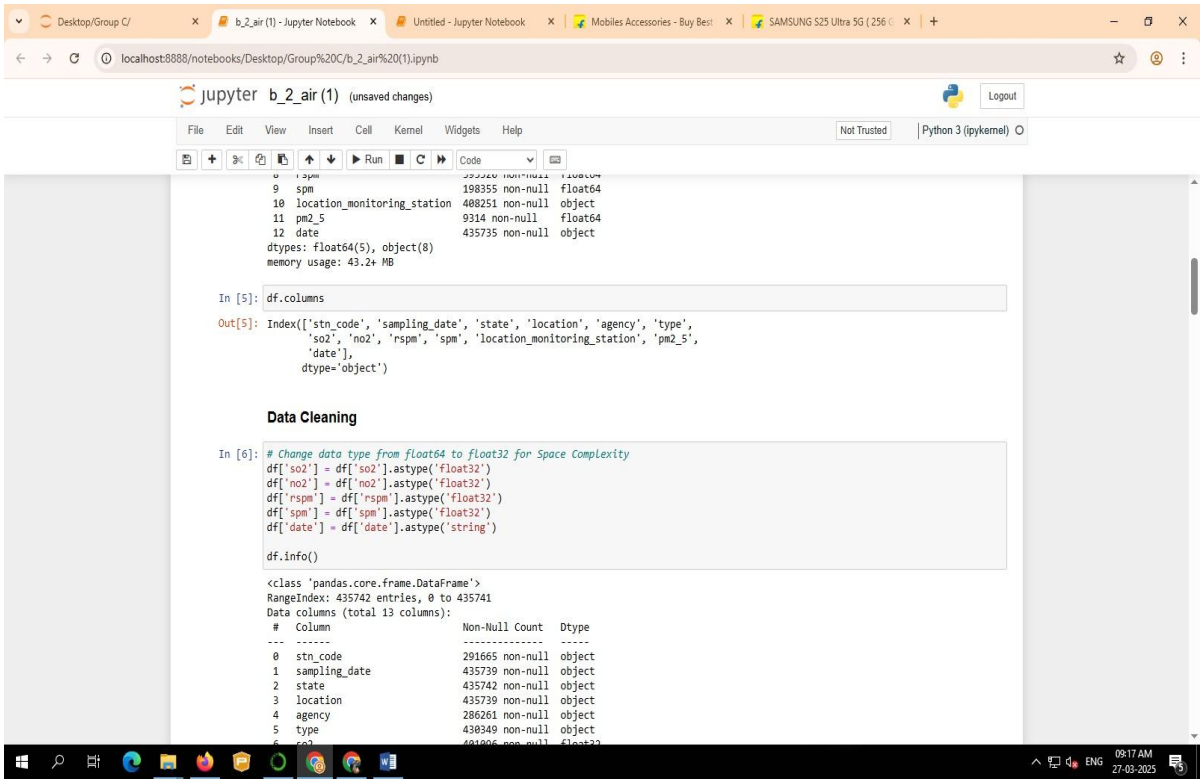
In [3]:
```
df.head()
```

Out[3]:

	stn_code	sampling_date	state	location	agency	type	so2	no2	rspm	spm	location_monitoring_station	pm2_5	date
0	150.0	February - M021990	Andhra Pradesh	Hyderabad	NaN	Residential, Rural and other Areas	4.8	17.4	NaN	NaN	NaN	NaN	1990-02-01
1	151.0	February - M021990	Andhra Pradesh	Hyderabad	NaN	Industrial Area	3.1	7.0	NaN	NaN	NaN	NaN	1990-02-01
2	152.0	February - M021990	Andhra Pradesh	Hyderabad	NaN	Residential, Rural and other Areas	6.2	28.5	NaN	NaN	NaN	NaN	1990-02-01
3	150.0	March - M031990	Andhra Pradesh	Hyderabad	NaN	Residential, Rural and other Areas	6.3	14.7	NaN	NaN	NaN	NaN	1990-03-01
4	151.0	March - M031990	Andhra Pradesh	Hyderabad	NaN	Industrial Area	4.7	7.5	NaN	NaN	NaN	NaN	1990-03-01

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 435742 entries, 0 to 435741
Data columns (total 13 columns):
 #   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
```



```
 9   spm                         198355 non-null  float64
 10  location_monitoring_station 408251 non-null  object
 11  pm2_5                       9314 non-null    float64
 12  date                        435735 non-null  object
dtypes: float64(5), object(8)
memory usage: 43.2+ MB
```

In [5]:
```
df.columns
```

Out[5]:
```
Index(['stn_code', 'sampling_date', 'state', 'location', 'agency', 'type',
       'so2', 'no2', 'rspm', 'spm', 'location_monitoring_station', 'pm2_5',
       'date'],
      dtype='object')
```

Data Cleaning

In [6]:
```
# Change data type from float64 to float32 for Space Complexity
df['so2'] = df['so2'].astype('float32')
df['no2'] = df['no2'].astype('float32')
df['rspm'] = df['rspm'].astype('float32')
df['spm'] = df['spm'].astype('float32')
df['date'] = df['date'].astype('string')

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 435742 entries, 0 to 435741
Data columns (total 13 columns):
 #   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
 0   stn_code                    291665 non-null  object
 1   sampling_date               435739 non-null  object
 2   state                       435742 non-null  object
 3   location                    435739 non-null  object
 4   agency                      286261 non-null  object
 5   type                        430349 non-null  object
```

Desktop/Group C/ | b_2_air (1) - Jupyter Notebook | Untitled - Jupyter Notebook | Mobiles Accessories - Buy Best | SAMSUNG S25 Ultra 5G (256 G

localhost:8888/notebooks/Desktop/Group%20C/b_2_air%20(1).ipynb

jupyter b_2_air (1) (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help | Not Trusted | Python 3 (ipykernel)

```
 3   location                    435739 non-null  object
 4   agency                      286261 non-null  object
 5   type                        430349 non-null  object
 6   so2                         401096 non-null  float32
 7   no2                         419509 non-null  float32
 8   rspm                        395520 non-null  float32
 9   spm                         198355 non-null  float32
 10  location_monitoring_station 408251 non-null  object
 11  pm2_5                       9314 non-null    float64
 12  date                        435735 non-null  string
dtypes: float32(4), float64(1), object(7), string(1)
memory usage: 36.6+ MB
```

In [7]:
```
df=df.drop_duplicates()
```

In [8]:
```
df.isna().sum()
```

Out[8]:
```
stn_code                     144077
sampling_date                     3
state                             0
location                          3
agency                       149466
type                           5357
so2                           34632
no2                           16222
rspm                          40035
spm                          236908
location_monitoring_station   27303
pm2_5                        425754
date                              7
dtype: int64
```

In [9]:
```
percent_missing = df.isnull().sum() * 100 / len(df)
```

In [10]:
```
percent_missing.sort_values(ascending=False)
```

Out[10]:
```
pm2_5                        97.859185
```

09:17 AM 27-03-2025

Desktop/Group C/ | b_2_air (1) - Jupyter Notebook | Untitled - Jupyter Notebook | Mobiles Accessories - Buy Best | SAMSUNG S25 Ultra 5G (256 G

localhost:8888/notebooks/Desktop/Group%20C/b_2_air%20(1).ipynb

jupyter b_2_air (1) (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help | Not Trusted | Python 3 (ipykernel)

```
date                         0.001069
sampling_date                0.000690
location                     0.000690
state                        0.000000
dtype: float64
```

In [11]:
```
df=df.drop(['stn_code', 'agency','sampling_date','location_monitoring_station','pm2_5'], axis = 1)
```

In [12]:
```
df.head()
```

Out[12]:

	state	location	type	so2	no2	rspm	spm	date
0	Andhra Pradesh	Hyderabad	Residential, Rural and other Areas	4.8	17.4	NaN	NaN	1990-02-01
1	Andhra Pradesh	Hyderabad	Industrial Area	3.1	7.0	NaN	NaN	1990-02-01
2	Andhra Pradesh	Hyderabad	Residential, Rural and other Areas	6.2	28.5	NaN	NaN	1990-02-01
3	Andhra Pradesh	Hyderabad	Residential, Rural and other Areas	6.3	14.7	NaN	NaN	1990-03-01
4	Andhra Pradesh	Hyderabad	Industrial Area	4.7	7.5	NaN	NaN	1990-03-01

In [13]:
```
df.columns
```

Out[13]:
```
Index(['state', 'location', 'type', 'so2', 'no2', 'rspm', 'spm', 'date'], dtype='object')
```

In [14]:
```
col_var = ['state', 'location', 'type','date']
col_num = ['so2','no2','rspm','spm']
```

In [15]:
```
for col in df.columns:
    if df[col].dtype == 'object' or df[col].dtype == 'string':
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].mean())
```

In [16]:
```
df.isna().sum()
```

Out[16]:
```
state       0
location    0
```

09:17 AM 27-03-2025

```
no2        0
rspm       0
spm        0
date       0
dtype: int64
```

In [17]:
```
df
```

Out[17]:

	state	location	type	so2	no2	rspm	spm	date
0	Andhra Pradesh	Hyderabad	Residential, Rural and other Areas	4.800000	17.400000	108.871712	220.774796	1990-02-01
1	Andhra Pradesh	Hyderabad	Industrial Area	3.100000	7.000000	108.871712	220.774796	1990-02-01
2	Andhra Pradesh	Hyderabad	Residential, Rural and other Areas	6.200000	28.500000	108.871712	220.774796	1990-02-01
3	Andhra Pradesh	Hyderabad	Residential, Rural and other Areas	6.300000	14.700000	108.871712	220.774796	1990-03-01
4	Andhra Pradesh	Hyderabad	Industrial Area	4.700000	7.500000	108.871712	220.774796	1990-03-01
...	...	...	...	...	...	...	...	...
435737	West Bengal	ULUBERIA	RIRUO	22.000000	50.000000	143.000000	220.774796	2015-12-24
435738	West Bengal	ULUBERIA	RIRUO	20.000000	46.000000	171.000000	220.774796	2015-12-29
435739	andaman-and-nicobar-islands	Guwahati	Residential, Rural and other Areas	10.830467	25.823299	108.871712	220.774796	2015-03-19
435740	Lakshadweep	Guwahati	Residential, Rural and other Areas	10.830467	25.823299	108.871712	220.774796	2015-03-19
435741	Tripura	Guwahati	Residential, Rural and other Areas	10.830467	25.823299	108.871712	220.774796	2015-03-19

435068 rows × 8 columns

In [18]:
```
df.isna().sum()
```

Out[18]:
```
state      0
location   0
type       0
so2        0
no2        0
rspm       0
spm        0
```

Data integration

In [19]:
```
subSet1 = df[['state', 'type']]
subSet2 = df[['state','location']]
```

In [20]:
```
subSet1.head()
```

Out[20]:

	state	type
0	Andhra Pradesh	Residential, Rural and other Areas
1	Andhra Pradesh	Industrial Area
2	Andhra Pradesh	Residential, Rural and other Areas
3	Andhra Pradesh	Residential, Rural and other Areas
4	Andhra Pradesh	Industrial Area

In [21]:
```
subSet2.head()
```

Out[21]:

	state	location
0	Andhra Pradesh	Hyderabad
1	Andhra Pradesh	Hyderabad
2	Andhra Pradesh	Hyderabad
3	Andhra Pradesh	Hyderabad
4	Andhra Pradesh	Hyderabad

In [22]:
```
concatenated_df = pd.concat([subSet1, subSet2], axis=1)
```

In [23]:
```
concatenated_df
```

Out[23]:

state	type	state	location

Desktop/Group C/ | b_2_air (1) - Jupyter Notebook | Untitled - Jupyter Notebook | Mobiles Accessories - Buy Best | SAMSUNG S25 Ultra 5G (256 G

localhost:8888/notebooks/Desktop/Group%20C/b_2_air%20(1).ipynb

jupyter b_2_air (1) (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help | Not Trusted | Python 3 (ipykernel)

In [23]:
```
concatenated_df
```

Out[23]:

	state	type	state	location
0	Andhra Pradesh	Residential, Rural and other Areas	Andhra Pradesh	Hyderabad
1	Andhra Pradesh	Industrial Area	Andhra Pradesh	Hyderabad
2	Andhra Pradesh	Residential, Rural and other Areas	Andhra Pradesh	Hyderabad
3	Andhra Pradesh	Residential, Rural and other Areas	Andhra Pradesh	Hyderabad
4	Andhra Pradesh	Industrial Area	Andhra Pradesh	Hyderabad
...	...	...	...	...
435737	West Bengal	RIRUO	West Bengal	ULUBERIA
435738	West Bengal	RIRUO	West Bengal	ULUBERIA
435739	andaman-and-nicobar-islands	Residential, Rural and other Areas	andaman-and-nicobar-islands	Guwahati
435740	Lakshadweep	Residential, Rural and other Areas	Lakshadweep	Guwahati
435741	Tripura	Residential, Rural and other Areas	Tripura	Guwahati

435068 rows × 4 columns

Error Correcting

In [24]:
```
def remove_outliers(column):
    Q1 = column.quantile(0.25)
    Q3 = column.quantile(0.75)
    IQR = Q3 - Q1
    threshold = 1.5 * IQR
    outlier_mask = (column < Q1 - threshold) | (column > Q3 + threshold)
    return column[~outlier_mask]
```

In [25]:
```
df.columns
```

Out[25]:
```
Index(['state', 'location', 'type', 'so2', 'no2', 'rspm', 'spm', 'date'], dtype='object')
```

ENG 09:17 AM 27-03-2025

Desktop/Group C/ | b_2_air (1) - Jupyter Notebook | Untitled - Jupyter Notebook | Mobiles Accessories - Buy Best | SAMSUNG S25 Ultra 5G (256 G

localhost:8888/notebooks/Desktop/Group%20C/b_2_air%20(1).ipynb

jupyter b_2_air (1) (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help | Not Trusted | Python 3 (ipykernel)

In [25]:
```
df.columns
```

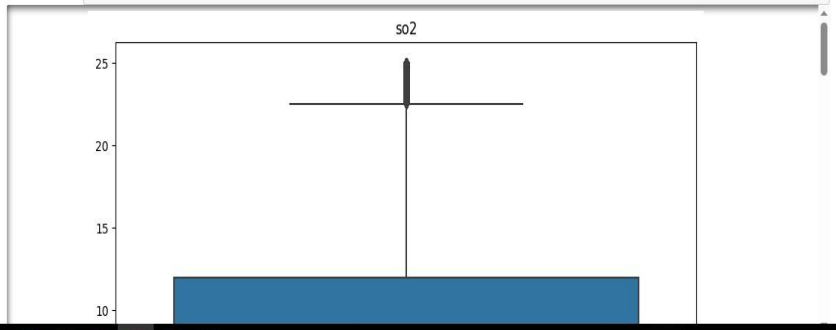
Out[25]:
```
Index(['state', 'location', 'type', 'so2', 'no2', 'rspm', 'spm', 'date'], dtype='object')
```

In [26]:
```
# Remove outliers for each column using a loop
col_name = ['so2', 'no2', 'rspm', 'spm']
for col in col_name:
    df[col] = remove_outliers(df[col])
```

In [27]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

In [28]:
```
plt.figure(figsize=(10, 6))  # Adjust the figure size if needed

for col in col_name:
    sns.boxplot(data=df[col])
    plt.title(col)
    plt.show()
```



ENG 09:17 AM 27-03-2025

jupyter b_2_air (1) Last Checkpoint: a few seconds ago (autosaved)

Data Transform

In [29]:
```
from sklearn.preprocessing import LabelEncoder

col_label= ['state','location','type']
# Initialize LabelEncoder

encoder = LabelEncoder()
# Iterate over columns
for col in df.columns:
        # Fit and transform the column
        df[col] = encoder.fit_transform(df[col])
```

In [30]:
```
df
```

Out[30]:

	state	location	type	so2	no2	rspm	spm	date
0	0	114	6	446	1489	2030	464	213
1	0	114	1	197	250	2030	464	213
2	0	114	6	790	3096	2030	464	213
3	0	114	6	823	1144	2030	464	214
4	0	114	1	427	301	2030	464	214
...	...	...	...	...	...	...	...	...
435737	35	282	3	2888	5307	2534	464	5059
435738	35	282	3	2809	5113	3098	464	5064
435739	36	100	6	1638	2696	2030	464	4779
435740	17	100	6	1638	2696	2030	464	4779
435741	31	100	6	1638	2696	2030	464	4779

435068 rows × 8 columns

In [ ]:

jupyter b_2_Heart (1) (unsaved changes)

In [23]:
```
# import pandas library
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score,confusion_matrix
from sklearn.linear_model import LogisticRegression
import seaborn as sns
import matplotlib.pyplot as plt
```

In [24]:
```
# Reading csv file
df = pd.read_csv("Heart.csv")
df.head()
```

Out[24]:

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

In [25]:
```
df.describe()
```

Out[25]:

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.00
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053	149.646865	0.326733	1.039604	1.399340	0.729373
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860	22.905161	0.469794	1.161075	0.616226	1.022606
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.00

Desktop/Group C/ | b_2_air (1) - Jupyter Notebo | b_2_Heart (1) - Jupyter Note | Untitled - Jupyter Notebook | Mobiles Accessories - Buy B | SAMSUNG S25 Ultra 5G (25

localhost:8888/notebooks/Desktop/Group%20C/b_2_Heart%20(1).ipynb

jupyter b_2_Heart (1) (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help | Not Trusted | Python 3 (ipykernel)

std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860	22.905161	0.469794	1.161075	0.616226	1.022606	0.61
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	0.0
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.500000	0.000000	0.000000	1.000000	0.000000	2.0
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	153.000000	0.000000	0.800000	1.000000	0.000000	2.0
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000	166.000000	1.000000	1.600000	2.000000	1.000000	3.0
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000	3.0

In [26]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trtbps    303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalachh  303 non-null    int64
 8   exng      303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slp       303 non-null    int64
 11  caa       303 non-null    int64
 12  thall     303 non-null    int64
 13  output    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

In [28]:
```
df1=df[df.duplicated()]
df1
```

ENG 09:19 AM 27-03-2025

Desktop/Group C/ | b_2_air (1) - Jupyter Notebo | b_2_Heart (1) - Jupyter Note | Untitled - Jupyter Notebook | Mobiles Accessories - Buy B | SAMSUNG S25 Ultra 5G (25

localhost:8888/notebooks/Desktop/Group%20C/b_2_Heart%20(1).ipynb

jupyter b_2_Heart (1) (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help | Not Trusted | Python 3 (ipykernel)

In [28]:
```
df1=df[df.duplicated()]
df1
```

Out[28]:

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
164	38	1	2	138	175	0	1	173	0	0.0	2	4	2	1

Data Cleaning

In [3]:
```
df = df.drop_duplicates()
```

In [4]:
```
# Count ,min,max ,etc of each column
df.describe()
```

Out[4]:

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	t
count	302.00000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.00
mean	54.42053	0.682119	0.963576	131.602649	246.500000	0.149007	0.526490	149.569536	0.327815	1.043046	1.397351	0.718543	2.314
std	9.04797	0.466426	1.032044	17.563394	51.753489	0.356686	0.526027	22.903527	0.470196	1.161452	0.616274	1.006748	0.613
min	29.00000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	0.00
25%	48.00000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.250000	0.000000	0.000000	1.000000	0.000000	2.00
50%	55.50000	1.000000	1.000000	130.000000	240.500000	0.000000	1.000000	152.500000	0.000000	0.800000	1.000000	0.000000	2.00
75%	61.00000	1.000000	2.000000	140.000000	274.750000	0.000000	1.000000	166.000000	1.000000	1.600000	2.000000	1.000000	3.00
max	77.00000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000	3.00

In [5]:
```
# Information about each column data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 302 entries, 0 to 302
Data columns (total 14 columns):
```

ENG 09:19 AM 27-03-2025

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 302 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       302 non-null    int64
 1   sex       302 non-null    int64
 2   cp        302 non-null    int64
 3   trtbps    302 non-null    int64
 4   chol      302 non-null    int64
 5   fbs       302 non-null    int64
 6   restecg   302 non-null    int64
 7   thalachh  302 non-null    int64
 8   exng      302 non-null    int64
 9   oldpeak   302 non-null    float64
 10  slp       302 non-null    int64
 11  caa       302 non-null    int64
 12  thall     302 non-null    int64
 13  output    302 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 35.4 KB
```

In [6]:
```
#Finding null values in each column
df.isna().sum()
```

Out[6]:
```
age         0
sex         0
cp          0
trtbps      0
chol        0
fbs         0
restecg     0
thalachh    0
exng        0
oldpeak     0
slp         0
caa         0
thall       0
```

Data Integration

In [7]:
```
df.head()
```

Out[7]:

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

In [8]:
```
df.fbs.unique()
```

Out[8]:
```
array([1, 0], dtype=int64)
```

In [9]:
```
subSet1 = df[['age','cp','chol','thalachh']]
```

In [10]:
```
subSet2 = df[['exng','slp','output']]
```

In [11]:
```
merged_df = subSet1.merge(right=subSet2,how='cross')
merged_df.head()
```

Out[11]:

	age	cp	chol	thalachh	exng	slp	output
0	63	3	233	150	0	0	1
1	63	3	233	150	0	0	1
2	63	3	233	150	0	2	1
3	63	3	233	150	0	2	1
4	63	3	233	150	1	2	1

3	63	3	233	150	0	2	1
4	63	3	233	150	1	2	1

Error Correcting

In [12]:
```
df.columns
```

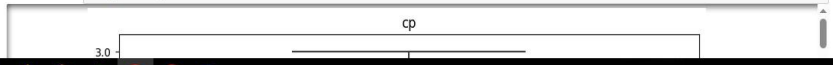
Out[12]:
```
Index(['age', 'sex', 'cp', 'trtbps', 'chol', 'fbs', 'restecg', 'thalachh',
       'exng', 'oldpeak', 'slp', 'caa', 'thall', 'output'],
      dtype='object')
```

In [13]:
```
def remove_outliers(column):
    Q1 = column.quantile(0.25)
    Q3 = column.quantile(0.75)
    IQR = Q3 - Q1
    threshold = 1.5 * IQR
    outlier_mask = (column < Q1 - threshold) | (column > Q3 + threshold)
    return column[~outlier_mask]
```

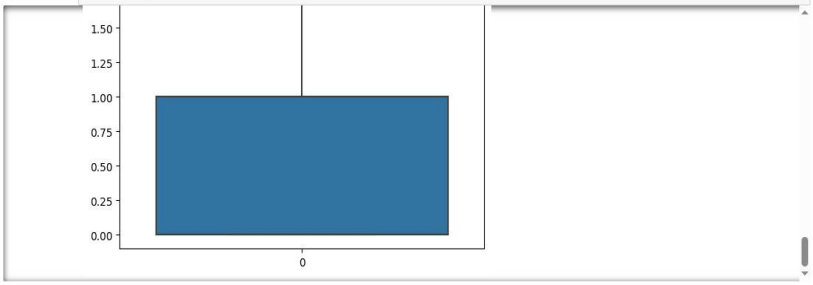
In [14]:
```
# Remove outliers for each column using a loop
col_name = ['cp','thalachh','exng','oldpeak','slp','caa']
for col in col_name:
    df[col] = remove_outliers(df[col])
```

In [15]:
```
plt.figure(figsize=(10, 6))  # Adjust the figure size if needed

for col in col_name:
    sns.boxplot(data=df[col])
    plt.title(col)
    plt.show()
```



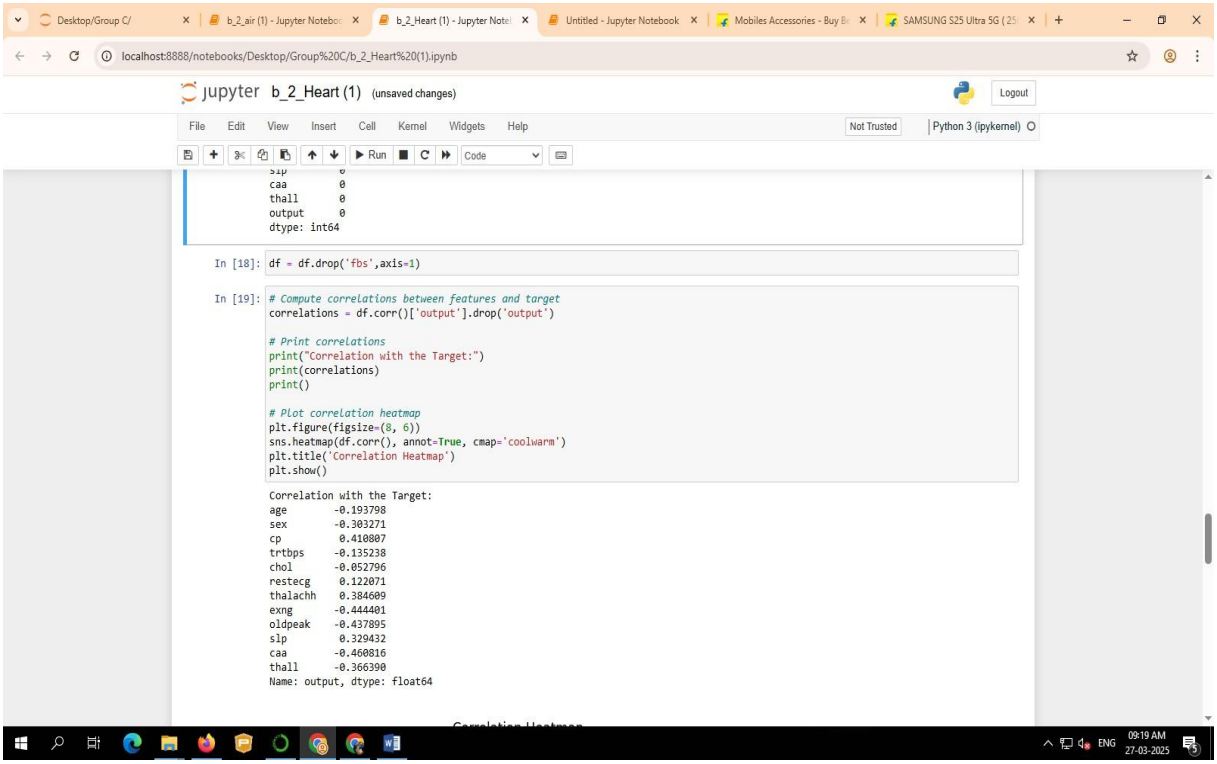

```
for col in col_name:
    sns.boxplot(data=df[col])
    plt.title(col)
    plt.show()
```



In [16]:
```
df = df.dropna()
```

In [17]:
```
df.isna().sum()
```

Out[17]:
```
age         0
sex         0
cp          0
trtbps      0
chol        0
fbs         0
restecg     0
thalachh    0
```



```
caa        0
thall      0
output     0
dtype: int64
```

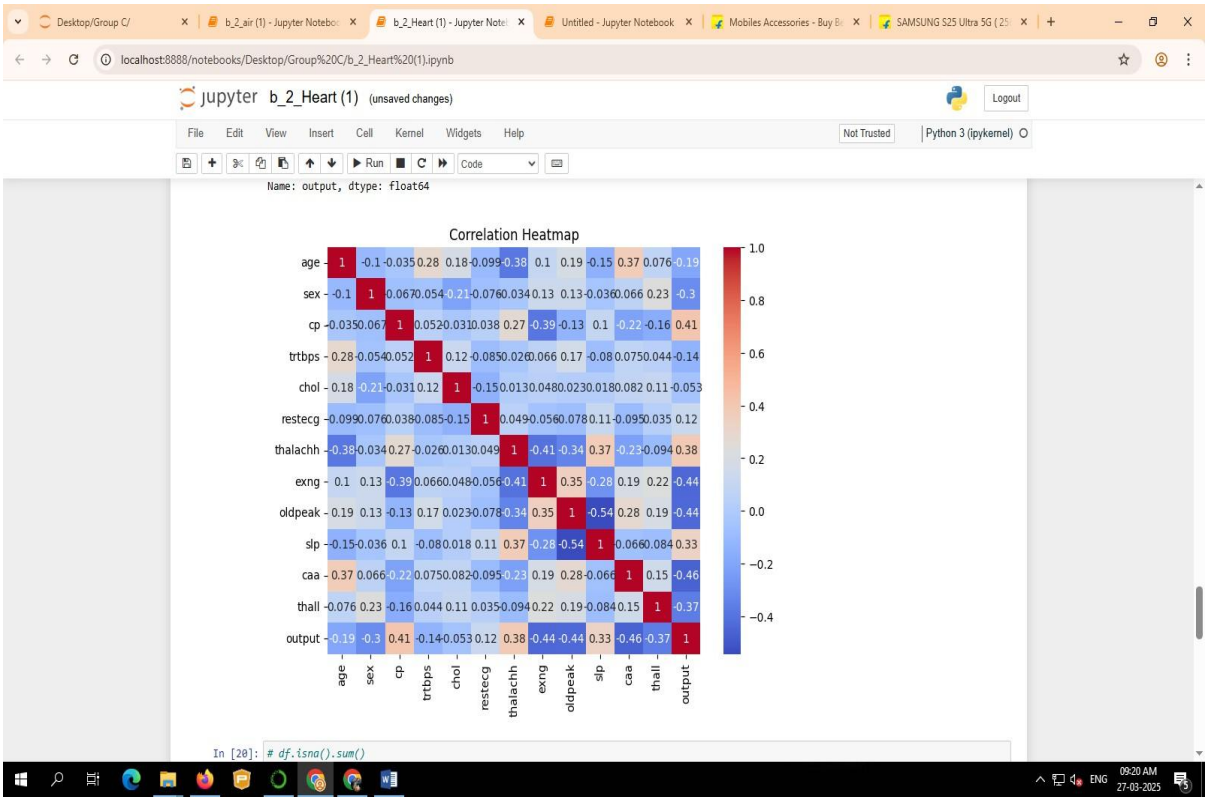
In [18]:
```
df = df.drop('fbs',axis=1)
```

In [19]:
```
# Compute correlations between features and target
correlations = df.corr()['output'].drop('output')

# Print correlations
print("Correlation with the Target:")
print(correlations)
print()

# Plot correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

```
Correlation with the Target:
age        -0.193798
sex        -0.303271
cp          0.410807
trtbps     -0.135238
chol       -0.052796
restecg     0.122071
thalachh    0.384609
exng       -0.444401
oldpeak    -0.437895
slp         0.329432
caa        -0.460816
thall      -0.366390
Name: output, dtype: float64
```



```
Name: output, dtype: float64
```


In [20]:
```
# df.isna().sum()
```

localhost:8888/notebooks/Desktop/Group%20C/b_2_Heart%20(1).ipynb

jupyter b_2_Heart (1) (unsaved changes)

In [20]:
```
# df.isna().sum()
```

Data Split

In [21]:
```
# splitting data using train test split
x = df[['cp','thalachh','exng','oldpeak','slp','caa']]
y = df.output
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=0)

x_train.shape,x_test.shape,y_train.shape,y_test.shape
```

Out[21]:
```
((220, 6), (55, 6), (220,), (55,))
```

Data transformation

In [22]:
```
from sklearn.preprocessing import StandardScaler
```

In [23]:
```
scaler = StandardScaler()
```

In [24]:
```
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

In [ ]:

Data model building

In [25]:
```
y_train= np.array(y_train).reshape(-1, 1)
y_test= np.array(y_test).reshape(-1, 1)
```

localhost:8888/notebooks/Desktop/Group%20C/b_2_Heart%20(1).ipynb

jupyter b_2_Heart (1) (unsaved changes)

In [25]:
```
y_train= np.array(y_train).reshape(-1, 1)
y_test= np.array(y_test).reshape(-1, 1)
```

In [26]:
```
y_train.shape
```

Out[26]:
```
(220, 1)
```

In [27]:
```
model = LogisticRegression()
model.fit(x_train_scaled, y_train)

# Make predictions on the test set
y_pred = model.predict(x_test_scaled)

# Evaluate the model's accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8363636363636363
```

```
C:\Users\Kumbh\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\utils\validation.py:1143: DataConversionWarnin
g: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using
ravel().
  y = column_or_1d(y, warn=True)
```

In [28]:
```
#Classification model using Decision Tree
from sklearn.tree import DecisionTreeClassifier
tc=DecisionTreeClassifier(criterion='entropy')
tc.fit(x_train_scaled,y_train)
y_pred=tc.predict(x_test_scaled)

print("Training Accuracy Score :",accuracy_score(y_pred,y_test))
print("Training Confusion Matrix  :",confusion_matrix(y_pred,y_test))
```

```
Training Accuracy Score : 0.8181818181818182
Training Confusion Matrix  : [[21  4]
 [ 6 24]]
```